Founder Fade Curves Do Not Predict Open-Source Project Survival

Abstract

Open-source software underpins critical global infrastructure, from Linux kernels to Python package ecosystems. Yet the sustainability of these projects remains fragile: roughly half of open-source projects that lose their primary maintainer cease active development within two years [1]. The dominant framework for studying this problem — Truck Factor Developer Detachment (TFDD) [1] — defines abandonment as the point when all developers holding significant codebase expertise become inactive. Projects are then classified as surviving if new core developers subsequently emerge, or collapsed if they do not.

The static snapshot approach has proven limited. Avelino et al. [1] found that among 1,932 popular GitHub projects, only 41 percent of those experiencing TFDD survived. Nourry et al. [2] replicated this on 36,464 projects and found an even lower 27 percent survival rate. Both studies identify static factors — project age, contributor count, bus factor, star count — as weak predictors, with little variance explained. As Nourry et al. note, the only metric showing a clear difference between surviving and non-surviving projects was project age at TFDD [2].

This limitation motivates searching for better predictors. Educational psychology offers a well-established framework for understanding how expertise transfers: Vygotsky's sociocultural theory [13] and Bruner, Wood, and Ross's scaffolding research [14, 15] demonstrate that expert learners internalize capabilities most effectively when support is gradually withdrawn rather than abruptly removed. In the open-source context, the founder's involvement — commits, merges, code reviews — constitutes a form of scaffolding: each decision they make models judgment for the community. A gradual decline in this involvement gives contributors repeated opportunities to observe, practice, and internalize decision-making. An abrupt departure, by contrast, leaves the community without the cognitive support needed to assume responsibility.

We test this scaffolding-fade hypothesis on real-world data. We ask three questions: (1) Do temporal fade descriptors of founder involvement outperform static project metrics in predicting whether a project survives its founder's departure? (2) Do projects with gradually fading founder involvement survive at higher rates than those with abrupt departures? (3) Does the fade curve of the founder predict survival better than the fade curve of other active contributors?

Our results disconfirm the hypothesis across all three questions. Using the ESEM 2019 dataset of 309 GitHub projects with founder departure events, we find that fade-only models perform below chance and that adding fade descriptors to static features yields no improvement. The directional effect reverses: collapsed projects have a slightly higher mean fade index than survived projects. A falsification control confirms that founder fade curves carry no genuine signal, performing worse than shuffled trajectories. The full quantitative results are presented in Section 4.

[FIGURE:fig1]

Our contributions are:

1. We rigorously test the scaffolding-fade hypothesis on real-world OSS data (309 projects from the ESEM 2019 dataset) and find it disconfirmed. 2. We define six quantitative fade descriptors extracted from monthly founder commit, merge, and review shares, and demonstrate their construction from public repository artifacts [1]. 3. We provide empirical evidence that

[1]Code: `https://github.com/ai-inventor-outputs/ai-invention-ad55a2-founder-fade-curve-predicts-oss-survival/tree/main/round-2/experiment-1`

fade descriptors perform below chance (AUC 0.462) and add no predictive value beyond static features, ruling out a theoretically motivated mechanism. 4. We identify that post-departure activity — not pre-departure founder trajectories — is the dominant predictor of survival, with commits after departure accounting for 33.5 percent of permutation importance. 5. We expand the related work to cover recent temporal analysis in OSS, including survival analysis with time-varying covariates, deep learning sequence models, and workflow dynamics analysis [2].

1 Introduction

Open-source software underpins critical global infrastructure, from Linux kernels to Python package ecosystems. Yet the sustainability of these projects remains fragile: roughly half of open-source projects that lose their primary maintainer cease active development within two years [1]. The dominant framework for studying this problem — Truck Factor Developer Detachment (TFDD) [1] — defines abandonment as the point when all developers holding significant codebase expertise become inactive. Projects are then classified as surviving if new core developers subsequently emerge, or collapsed if they do not.

The static snapshot approach has proven limited. Avelino et al. [1] found that among 1,932 popular GitHub projects, only 41 percent of those experiencing TFDD survived. Nourry et al. [2] replicated this on 36,464 projects and found an even lower 27 percent survival rate. Both studies identify static factors — project age, contributor count, bus factor, star count — as weak predictors, with little variance explained. As Nourry et al. note, the only metric showing a clear difference between surviving and non-surviving projects was project age at TFDD [2].

This limitation motivates searching for better predictors. Educational psychology offers a well-established framework for understanding how expertise transfers: Vygotsky's sociocultural theory [13] and Bruner, Wood, and Ross's scaffolding research [14, 15] demonstrate that expert learners internalize capabilities most effectively when support is gradually withdrawn rather than abruptly removed. In the open-source context, the founder's involvement — commits, merges, code reviews — constitutes a form of scaffolding: each decision they make models judgment for the community. A gradual decline in this involvement gives contributors repeated opportunities to observe, practice, and internalize decision-making. An abrupt departure, by contrast, leaves the community without the cognitive support needed to assume responsibility.

We test this scaffolding-fade hypothesis on real-world data. We ask three questions: (1) Do temporal fade descriptors of founder involvement outperform static project metrics in predicting whether a project survives its founder's departure? (2) Do projects with gradually fading founder involvement survive at higher rates than those with abrupt departures? (3) Does the fade curve of the founder predict survival better than the fade curve of other active contributors?

Our results disconfirm the hypothesis across all three questions. Using the ESEM 2019 dataset of 309 GitHub projects with founder departure events, we find that fade-only models perform below chance and that adding fade descriptors to static features yields no improvement. The directional effect reverses: collapsed projects have a slightly higher mean fade index than survived projects. A falsification control confirms that founder fade curves carry no genuine signal, performing worse than shuffled trajectories. The full quantitative results are presented in Section 4.

[FIGURE:fig1]

Our contributions are:

1. We rigorously test the scaffolding-fade hypothesis on real-world OSS data (309 projects from the ESEM 2019 dataset) and find it disconfirmed. 2. We define six quantitative fade descrip-

[2]Code: `https://github.com/ai-inventor-outputs/ai-invention-ad55a2-founder-fade-curve-predicts-oss-survival/tree/main/round-2/research-1`

tors extracted from monthly founder commit, merge, and review shares, and demonstrate their construction from public repository artifacts [3]. 3. We provide empirical evidence that fade descriptors perform below chance (AUC 0.462) and add no predictive value beyond static features, ruling out a theoretically motivated mechanism. 4. We identify that post-departure activity — not pre-departure founder trajectories — is the dominant predictor of survival, with commits after departure accounting for 33.5 percent of permutation importance. 5. We expand the related work to cover recent temporal analysis in OSS, including survival analysis with time-varying covariates, deep learning sequence models, and workflow dynamics analysis [4].

2 Related Work

2.1 Open-Source Abandonment and Survival

The foundational work on open-source project survival is Avelino et al.'s Truck Factor Developer Detachment framework [1]. They define the truck factor as the minimum number of developers whose simultaneous departure would seriously impair a project, computed using the Degree of Authorship metric [16]. A TFDD event occurs when all truck-factor developers become inactive, defined as one year without commits. Among 1,932 popular GitHub projects, 16 percent experienced TFDD and 41 percent of those survived by attracting at least one new truck-factor developer [1]. Surviving projects tended to be younger at TFDD, have more post-departure commits, and attract a single new core developer in 86 percent of cases.

Nourry et al. [2] replicated this on 36,464 projects including smaller, less popular ones, and found dramatically different rates: 89.6 percent faced TFDD but only 27 percent survived. The disparity is explained by sample composition — smaller projects lack the community gravity to attract new maintainers. Nourry et al. found that project age at TFDD was the only static metric showing a clear difference between survivors and non-survivors.

Other work has examined core developer turnover patterns. Calefato et. [3] found that 45 percent of core developers disengage for at least one year, with 35 to 55 percent returning. Jamieson et al. [4] showed that value-related discussions in GitHub issues predict contributor turnover, suggesting that social dynamics matter beyond pure code metrics.

2.2 Founder and Governance Dynamics

Noori et al. [5] applied natural-language processing to GOVERNANCE.md files across 637 repositories to characterize how textual governance evolves as projects mature. They documented institutional maturation but did not predict survival outcomes. Their work differs from ours in modality — textual governance rather than behavioral trajectory — and in outcome — descriptive rather than predictive.

Chen et al. [6] used difference-in-differences across 50,804 repositories to estimate the impact of core contributor disengagement on pull-request throughput, acceptance rates, and merge time. They found that impact varies with static contributor profiles but did not model the founder specifically or predict survival. (This work is cited from the hypothesis literature review; the full publication details remain pending.)

[3]Code: `https://github.com/ai-inventor-outputs/ai-invention-ad55a2-founder-fade-curve-predicts-oss-survival/tree/main/round-2/experiment-1`

[4]Code: `https://github.com/ai-inventor-outputs/ai-invention-ad55a2-founder-fade-curve-predicts-oss-survival/tree/main/round-2/research-1`

2.3 Temporal Methods for OSS Prediction

Recent work has explored temporal approaches to OSS sustainability prediction. Karim et al. [7] built a hierarchical Transformer model over 24-month aggregate activity sequences to classify projects into lifecycle stages, achieving over 94 percent accuracy. Their work covers aggregate temporal patterns but does not isolate founder involvement trajectories or predict post-departure survival. The temporal modeling literature also includes survival analysis with time-varying covariates [8], typically using project-level aggregates like commit frequency, and deep learning sequence models (LSTM/GRU/Transformer) modeling aggregate activity patterns [7].

Kaushik and Chahal [9] identified a death spiral in open-source projects through pull-request workflow dynamics — increasing friction, backlog growth, falling innovation, and rising merge latency. Their analysis is post-hoc, beginning after decline starts, and is community-level rather than founder-specific. They note that popularity and innovation are causes of survival while workflow friction is a byproduct, but do not analyze the founder's behavioral trajectory before departure.

2.4 Scaffolding and Fading in Education

The concept of scaffolding originates in Vygotsky's sociocultural theory [13], which posits that learning occurs within a Zone of Proximal Development — the space between what a learner can do independently and what they can achieve with guidance. Bruner, Wood, and Ross [14] operationalized this as scaffolding: a tutor provides structured support that is gradually withdrawn as the learner internalizes the skill. Wood et. [15] demonstrated that optimal learning occurs when support is reduced incrementally; abrupt removal before competence matures causes performance collapse.

This educational mechanism has been replicated across domains including mathematics education, programming education, and second-language acquisition, but has never been applied to open-source sustainability. Our contribution is the cross-domain transfer attempt: we treat the founder's involvement as scaffolding and test whether the shape of the fade curve predicts post-departure survival. Our results show that this transfer does not hold in the OSS context.

3 Methods

3.1 Problem Definition

We study the prediction of open-source project survival after founder departure. Let P be an open-source project with founder f. Let T be the set of monthly time points from project inception to founder departure, where n is the number of months observed.

For each month t_i, we define three involvement measures for the founder:

- $C(t_i)$: founder's share of total commits in month t_i - $M(t_i)$: founder's share of total pull-request merges in month t_i - $R(t_i)$: founder's share of total code-review decisions in month t_i

The founder's combined involvement at time t_i is the average of these three shares:

$$S(t_i) = \frac{C(t_i) + M(t_i) + R(t_i)}{3}$$

We define the founder fade curve as the time series of S values over the pre-departure window.

The founder departs at time t_n, defined as a 12-month inactivity window from the last commit, consistent with the Avelino et al. criterion [1]. We label the project as surviving if at least one new truck-factor developer appears with sustained activity in the 24 months post-departure, following the ESEM 2019 criterion [1]. Otherwise, the project is labeled collapsed.

3.2 Fade Descriptors

We extract six quantitative descriptors from the fade curve. All curves are denoised using a Savitzky-Golay filter with window length five and polynomial order two before computing descriptors, following signal-processing best practices for noisy time-series data.

1. **Linear slope**: The slope of a linear regression of S on time, normalized by the initial value $S(t_0)$. Negative slope indicates gradual decline; positive slope indicates increasing involvement.
2. **Convexity**: The leading coefficient of a quadratic fit to the smoothed curve, normalized by $S(t_0)$, capturing whether the fade accelerates or decelerates.
3. **Decline onset time**: The first month where the smoothed first derivative is consistently negative (below -0.01), measured as a fraction of total months from project start.
4. **Cliff score**: The ratio of the final two-month drop to the average of the preceding six months, bounded between 0 and 1. High values indicate abrupt departure.
5. **Plateau indicator**: A binary flag indicating whether the curve maintained low variance for at least five months before the decline onset, suggesting a plateau-then-cliff pattern.
6. **Fade index**: A composite score bounded between 0 and 1, where 1 indicates a smooth linear fade and 0 indicates an abrupt cliff. Computed as $1 - \text{cliff\_score} + 0.3$ if slope is negative, minus 0.2 if plateau is detected.

3.3 Static Features

We compare fade descriptors against seven static features measured at departure:

- Bus factor: Minimum number of developers whose departure would impair the project [16].
- Contributor count: Total number of unique contributors at departure.
- Stars (log-transformed): GitHub star count at departure.
- File count (log-transformed): Number of files in the repository at departure.
- Repository age: Years from repository creation to departure.
- Commits before departure (log-transformed): Total commits in the pre-departure period.
- Commits after departure (log-transformed): Total commits in the 24-month post-departure period.

3.4 Data Sources

We use the ESEM 2019 dataset [1], which provides 315 GitHub projects with TFDD events, sourced from Zenodo (10.5281/zenodo.2546008). After filtering for projects with at least 6 months of pre-departure trajectory data, we obtain 309 projects: 127 survived and 182 collapsed. The dataset includes monthly time-series features for founder commit, merge, and review shares, along with static project metadata (stars, forks, contributors, bus factor).

3.5 Experimental Setup

We train four models using stratified five-fold cross-validation:

- **Model A**: Logistic regression with static features only.
- **Model B**: Logistic regression with fade descriptors only.
- **Model C**: Logistic regression with all features combined (static + fade + interaction terms).
- **Model D**: Random forest with all features combined.

We evaluate using area under the ROC curve for classification and log-loss for probability calibration. Permutation importance assesses feature contribution. For directionality analysis, we compare mean fade descriptors between survived and collapsed groups using independent two-sample t-tests with Cohen's d effect sizes. A falsification control replaces the founder's fade index with uniformly random values and retrains the fade-only model.

4 Experiments and Results

4.1 Main Results

Table 1 summarizes the cross-validated performance of all models on the 309-project ESEM 2019 dataset.

Table 1: Model Performance on 309 Projects

Model	Features	AUC	AUC St
A (Static)	bus_factor, contributors, age, stars, files, commits_before, commits_after	0.928	0.029
B (Fade)	slope, convexity, onset, cliff, plateau, fade_idx	0.462	0.091
C (Combined)	All features + interactions	0.929	0.030
D (Random Forest)	All features + interactions	0.880	0.032

[FIGURE:fig2]

Model B achieves AUC of 0.462, which is below the chance level of 0.500. This indicates that fade descriptors not only fail to predict survival — they predict it in the wrong direction. Model C achieves AUC of 0.929, essentially identical to Model A's 0.928, confirming that fade descriptors add no meaningful predictive value beyond static features. The random forest model (Model D) achieves lower AUC (0.880), suggesting that the relationship between features and survival is approximately linear and that the logistic regression captures it well.

The static-only model's strong performance (AUC 0.928) is driven primarily by post-departure activity. The feature log-transformed commits after departure accounts for 33.5 percent of permutation importance in the combined model, followed by the interaction term the interaction between fade index and contributor count (16.8 percent) and contributor count (12.7 percent). Pure fade descriptors rank at the bottom: cliff score (0.09 percent) and fade index (-0.007 percent).

4.2 Directionality

The directionality analysis reveals that the predicted relationship between fade patterns and survival does not hold in the data. The mean fade index for survived projects is 0.934, while collapsed

Table 2: Permutation Feature Importance (Combined Model)

Feature	Importance
commits_after_log	0.335
fade_idx_x_contributors	0.168
contributor_count	0.127
commits_before_log	0.125
bus_factor	0.052
cliff_x_bus_factor	0.002
stars_log	0.001
file_count_log	0.001
S_cliff	0.001
S_fade_idx	-0.000

projects have a mean of 0.962 — opposite to the predicted direction. The difference is not statistically significant (t = -1.329, p = 0.185, Cohen's d = -0.154).

The cliff score shows a similar pattern: survived projects have a mean cliff score of 0.111 versus 0.076 for collapsed projects (p = 0.231), again opposite to the predicted direction. The only descriptor showing a significant difference is the normalized slope: survived projects have a mean slope of -0.0114 versus -0.0086 for collapsed projects (p = 0.0009), with survived projects showing a slightly more negative (steeper declining) slope. However, this effect is small and its interpretation is ambiguous — a steeper decline could indicate either more aggressive fading or a sharper pre-departure withdrawal.

[FIGURE:fig3]

4.3 Falsification Control

To test whether founder fade curves carry any genuine signal, we replace the founder's fade index with uniformly random values drawn from [0, 1] and retrain the fade-only model. The shuffled model achieves AUC of 0.536, compared to 0.462 for the actual founder fade curve. The difference of -0.074 indicates that the founder's actual fade trajectory performs worse than random noise. This falsification control confirms that the fade descriptors do not encode any genuine predictive signal about project survival.

5 Discussion

5.1 Interpretation

Our results disconfirm the scaffolding-fade hypothesis in the open-source context. Despite the theoretical plausibility that gradual founder involvement reduction would scaffold community capability, the empirical data shows no such effect. Three findings are notable:

First, fade descriptors perform below chance (AUC 0.462), suggesting that the relationship between founder involvement trajectories and project survival is either absent or operates in a direction opposite to our hypothesis. The slight reversal in directionality (collapsed projects having higher fade indices) may reflect confounding factors: projects that survive may do so because they are larger and more active, and larger projects naturally have flatter founder involvement curves (the founder's share of activity is diluted across more contributors), producing higher fade indices by construction rather than by design.

Second, the dominant predictor of survival is post-departure activity (log-transformed commits after departure, importance 0.335). This suggests that survival is determined by what happens after the founder leaves — whether the community continues to contribute — rather than by what happened before. The static features that drive prediction are largely measures of project momentum and community size, not of the founder's departure pattern.

Third, the interaction term the interaction between fade index and contributor count ranks second in importance (0.168), suggesting that the relationship between fade patterns and survival may depend on project size. However, this interaction does not rescue the main effect: in isolation, fade descriptors remain uninformative.

5.2 Why the Hypothesis Failed

Several factors may explain why the scaffolding-fade hypothesis does not hold in open-source contexts:

Different mechanism of knowledge transfer: Educational scaffolding operates through deliberate, structured support withdrawal. Open-source founder involvement is rarely a planned fade — it is typically driven by external factors (career changes, burnout, personal circumstances) rather than intentional capability transfer. The founder's declining involvement may reflect personal constraints rather than community readiness.

Implicit vs. explicit scaffolding: In education, scaffolding is explicit and observable. In open-source projects, the founder's "scaffolding" — if it exists — is implicit in the codebase architecture, documentation, and community norms. These may transfer regardless of the founder's activity trajectory.

Community self-organization: Open-source communities may self-organize around capability transfer through mechanisms not captured by founder activity curves — issue discussions, pull-request reviews by non-founders, documentation contributions, and informal mentorship.

Selection effects: Projects that survive may do so because of inherent characteristics (topic relevance, ecosystem position, funding) that are independent of founder behavior. The founder's involvement trajectory may be a byproduct of these characteristics rather than a cause of survival.

5.3 Comparison to Prior Work

Our results are consistent with Nourry et al.'s finding that static metrics explain little variance in survival [2], but they extend that finding to temporal descriptors. Where Nourry et al. found that project age was the only static metric showing clear separation, we find that post-departure activity (a retrospective measure) is the strongest predictor. This suggests that survival is determined by post-departure dynamics rather than pre-departure conditions.

Our results contrast with Karim et al.'s success in predicting lifecycle stages using temporal sequences [7]. The difference may be that lifecycle stage classification is a different task from survival prediction: the former classifies current state from aggregate patterns, while the latter predicts a binary outcome from a specific event. Our fade descriptors target a narrower signal (founder-specific trajectory) that may be too weak to carry predictive value.

5.4 Practical Implications

For open-source maintainers: The absence of a fade effect suggests that conscious fading of involvement is not a proven survival strategy. Instead, maintainers should focus on building community capacity through explicit mechanisms: documentation, onboarding processes, and distributed decision-making — regardless of their own activity trajectory.

For ecosystem funders: Evaluating fade trajectories is not a reliable triage signal. Resources should be directed toward projects based on community size, post-departure activity, and ecosystem position — the features that actually predict survival.

5.5 Limitations

Our study has several limitations. The 309-project dataset, while drawn from real-world data, is modest in size and may lack power to detect small effects. Our founder identification relies on repository creation metadata and earliest sustained contribution, which may misidentify founders in projects with early co-founders. Our analysis is restricted to GitHub artifacts and may not generalize to other platforms. We use the Avelino et al. criterion for survival, which may misclassify projects that survive through distributed maintenance without a single new core developer. Our observational analysis cannot establish causality, and the negative results could reflect measurement error in the fade descriptors rather than absence of the underlying mechanism.

6 Conclusion

We tested the scaffolding-fade hypothesis — that the shape of a founder's involvement trajectory predicts open-source project survival after departure — on the ESEM 2019 dataset of 309 GitHub projects. The hypothesis is disconfirmed: fade-only models achieve AUC of 0.462 (below chance), adding fade descriptors to static features yields no improvement (combined AUC 0.929 versus static-only AUC 0.928), and the directional effect reverses (collapsed projects have higher mean fade index). A falsification control confirms that founder fade curves carry no genuine signal.

The primary predictor of survival is post-departure activity, not pre-departure founder trajectories. This suggests that open-source project survival is determined by what happens after the founder leaves — whether the community continues to contribute — rather than by how the founder departed. The scaffolding-fade mechanism from educational psychology, while theoretically plausible, does not operate in the open-source context as hypothesized.

Future work should explore alternative mechanisms of capability transfer in open-source communities, including explicit governance structures, documentation quality, and community self-organization patterns. The negative result reported here rules out one specific mechanism and sharpens the question of what actually enables open-source projects to survive their founders.